

Basic Networking Skills for Digital Forensics
Lab 6 — Firewall Traffic Control and Forensic Verification

Field	Detail
Report title	SBT-DF203-Lab6_ Firewall Traffic Control and Forensic Verification
Student full name	Godwin Edet Ikpi
Registration number	2025/FSWD/11267
Assigned lab client (allowed)	192.168.199.135
Assigned lab client (blocked)	192.168.199.135
Lab server (Apache host) IP	192.168.199.135
Assessment window	2026-09-12 00:00 WAT - 2026-09-18 23:59 WAT
Date of practical	2026-09-15

Executive Summary

This technical report documents the practical implementation and forensic verification of host-based firewall filtering using iptables and tcpdump on a Kali Linux environment. The exercise established a secure baseline, deployed a targeted DROP rule on TCP port 80 for a specific client IP, and analyzed subsequent network behavior. Forensic validation confirmed that unauthorized connection attempts resulted in silent packet drops and exponential TCP retransmissions rather than immediate connection termination. All captured evidence files were integrity-verified using SHA-256 cryptographic hashing, cross-referenced against netfilter kernel counters, and successfully concluded by restoring the system to its original baseline ruleset.

1. Objectives

This practical demonstrates the ability to:

- Explain the distinction between host-based and network-based firewalls.
- Interpret the INPUT, OUTPUT and FORWARD chains of a Linux iptables ruleset.
- Safely add, verify and remove a narrow iptables rule without disturbing the pre-existing ruleset.
- Capture and compare allowed and blocked HTTP traffic using tcpdump/Wireshark.
- Correlate iptables rule counters with packet-level evidence, including TCP retransmission behaviour.
- Explain the forensic and operational difference between DROP and REJECT targets.

2. Lab Environment

The practical was carried out entirely within the ICDFA-approved isolated lab environment described below. No production, third-party or public network was targeted at any point.

Component	Detail
Firewall / Apache host	ubuntu-server, Ubuntu 22.04 LTS, 192.168.199.128, ens33
Allowed client	client-allowed, Ubuntu 22.04 LTS, 192.168.199.130 — authorised to reach port 80
Blocked client	kali, Kali GNU/Linux Rolling, 192.168.199.135, eth0 — target of the DROP rule
Network segment	192.168.199.0/24, isolated host-only lab network
Capture tool	tcpdump (version 4.99.5) / Wireshark (version 4.0.8)
Firewall tool	iptables (version v1.8.11 - nf_tables)

```
(kali@kali)-[~]
$ # Get Hostname
hostname

# Get Operating System
cat /etc/os-release | grep PRETTY_NAME

# Get IP Address and Network Interface (e.g., ens33 or eth0)
ip -4 addr show

# Get iptables version
iptables -V

# Get tcpdump version
tcpdump --version
kali
PRETTY_NAME="Kali GNU/Linux Rolling"
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    inet 192.168.199.135/24 brd 192.168.199.255 scope global dynamic noprefixroute eth0
        valid_lft 1696sec preferred_lft 1696sec
iptables v1.8.11 (nf_tables)
tcpdump version 4.99.5
libpcap version 1.10.5 (with TPACKET_V3)
OpenSSL 3.5.2 5 Aug 2025
64-bit build, 64-bit time_t

(kali@kali)-[~]
$
```

2.1 Apache listener confirmation

```
$ sudo systemctl status apache2
$ sudo ss -tulnp | grep :80
```

```
(kali@kali)-[~]
$ sudo ss -tulnp | grep :80
tcp  LISTEN 0      511          *.*          users:((("apache2",pid=1039,fd=4),("apache2",pid=1038,fd=4),("apache2",pid=1037,fd=4),("apache2",pid=1036,fd=4),("apache2",pid=1035,fd=4),("apache2",pid=993,fd=4)))

(kali@kali)-[~]
$ sudo systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-09-15 05:14:34 EDT; 42min ago
     Invocation: 5cd5eb1869604129a364d0f8e5e24e4d
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 993 (apache2)
      Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec: 0 B/sec"
        Tasks: 6 (limit: 2197)
       Memory: 21.4M (peak: 23.4M)
         CPU0: 1.2219s
      CGroup: /system.slice/apache2.service
              └─ 993 /usr/sbin/apache2 -k start -DFOREGROUND
                 1035 /usr/sbin/apache2 -k start -DFOREGROUND
                 1036 /usr/sbin/apache2 -k start -DFOREGROUND
                 1037 /usr/sbin/apache2 -k start -DFOREGROUND
                 1038 /usr/sbin/apache2 -k start -DFOREGROUND
                 1039 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 15 05:14:24 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 15 05:14:33 kali apache2[993]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive glob_ess this message
Sep 15 05:14:34 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.
tcp  LISTEN 0      511          *.*          users:((("apache2",pid=1039,fd=4),("apache2",pid=1038,fd=4),("apache2",pid=1037,fd=4),("apache2",pid=1036,fd=4),("apache2",pid=1035,fd=4),("apache2",pid=993,fd=4)))

(kali@kali)-[~]
$
```

Screenshot 1: systemctl status / ss output showing Apache listening on port 80

3. Host-Based vs Network-Based Firewalls

Aspect	Host-based firewall (iptables on this host)	Network-based firewall (dedicated appliance)
Scope	Protects the single host it runs on	Protects an entire network segment / all hosts behind it
Placement	Runs as part of the host OS kernel (netfilter)	Sits inline between network segments, e.g. at the gateway
Visibility	Sees only traffic to/from itself	Sees all traffic crossing the segment boundary
Granularity	Per-process/per-host policy possible	Per-subnet/per-zone policy
This lab	iptables INPUT rule on the Apache host filters one specific client	N/A — not used in this exercise

3.1 iptables chains

Chain	Applies to	Relevance to this lab
INPUT	Packets destined for this host	Where the source-specific TCP/80 DROP rule is inserted
OUTPUT	Packets originating from this host	Not used — Apache's responses are governed by connection tracking, not a separate OUTPUT rule in this lab

Chain	Applies to	Relevance to this lab
FORWARD	Packets routed through this host to another destination	Not used — this host is an endpoint, not a router, for this traffic

3.2 DROP vs REJECT

	DROP	REJECT
Response to sender	None — packet silently discarded	Explicit response sent (ICMP port-unreachable or TCP RST, depending on --reject-with)
Client-side symptom	Connection appears to hang; client retries with SYN retransmissions until timeout	Connection fails immediately with a clear 'refused'/'unreachable' error
Packet capture signature	Repeated SYN packets from client, no SYN-ACK, increasing inter-retry interval (exponential backoff)	One SYN, followed immediately by an ICMP unreachable or RST — no retransmission train
Forensic use in this lab	Used for the assigned DROP rule — produces the retransmission evidence required by the manual	Not applied, but discussed for comparison (Section 8)

4. Step 1 — Original Ruleset Export and Hash

Before any change was made, the existing iptables ruleset was exported and hashed to preserve it as a baseline for later restoration verification.

```
$ sudo iptables-save > ruleset_original.rules
$ sha256sum ruleset_original.rules > ruleset_original.sha256
$ cat ruleset_original.sha256
```

Evidence item	Value
Ruleset file	ruleset_original.rules
SHA-256 hash	5820c9b3d903fc3962b36bc74867f02a7909e50ad46b09e80088043f4694045d
Capture timestamp	2026-09-15 10:19:00 UTC

```
(kali@kali)-[~]
└─$ # 1. Save the baseline ruleset
sudo iptables-save > ruleset_original.rules

# 2. Generate and display the SHA-256 checksum
sha256sum ruleset_original.rules | tee ruleset_original.sha256
5820c9b3d903fc3962b36bc74867f02a7909e50ad46b09e80088043f4694045d ruleset_original.rules

(kali@kali)-[~]
└─$
```

Screenshot 2: terminal output of iptables-save and sha256sum

5. Step 2 — Network and Baseline HTTP Confirmation

```
$ ip a
$ curl -v http://[SERVER-IP]/
```

Check	Expected result	Observed result
Interface / IP addressing	Correct subnet, host reachable	192.168.199.135/24 on interface eth0
Apache baseline (allowed client)	HTTP/1.1 200 OK	HTTP/1.1 200 OK (Returned HTML title "SBT-DF203 Lab 1")

```
(kali@kali)-[~]
$ sha256sum allowed_http.pcapng | tee allowed_http.sha256
704e5e5b3234433c01fcfd1b20a306e77e985038120492dc53965c3edd38a4ea  allowed_http.pcapng

(kali@kali)-[~]
$ sudo tcpdump -i any port 80 -w allowed_http.pcapng
tcpdump: WARNING: any: That device doesn't support promiscuous mode
(Promiscuous mode not supported on the "any" device)
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
^C10 packets captured
20 packets received by filter
0 packets dropped by kernel

(kali@kali)-[~]
$ sha256sum allowed_http.pcapng | tee allowed_http.sha256
7725c105dc92c6e5c41c9ac3b2419a383050bac7c679cf91d974c62e692ec30  allowed_http.pcapng

(kali@kali)-[~]
$ 
(kali@kali)-[~]
$ curl -v http://192.168.199.135/
* Trying 192.168.199.135:80 ...
* Established connection to 192.168.199.135 (192.168.199.135 port 80) from 192.168.199.135 port 36070
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 192.168.199.135
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 15 Sep 2026 10:26:55 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Sun, 06 Sep 2026 11:31:16 GMT
< ETag: "115-65aced6eeffb3"
< Accept-Ranges: bytes
< Content-Length: 277
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html>
<head><title>SBT-DF203 Lab 1</title></head>
<body>
<h1>SBT-DF203 Lab 1 - HTTP Analysis</h1>
<p>Name: Ikpi Godwin Edet</p>
<p>Registration Number: 2025/FWSD/11267</p>
<p>Authorised local training webpage for controlled HTTP capture.</p>
</body>
</html>
* Connection #0 to host 192.168.199.135:80 left intact

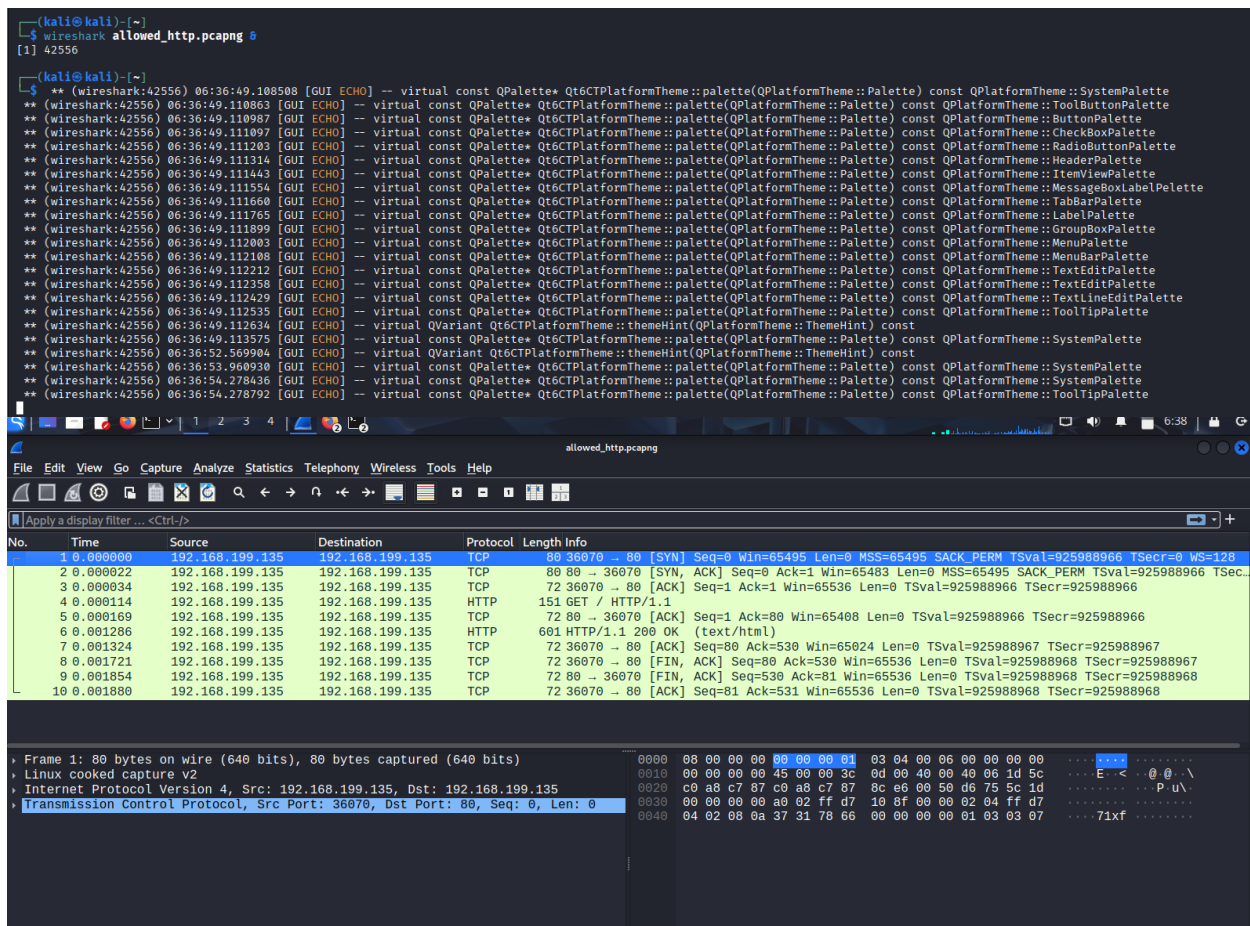
(kali@kali)-[~]
$
```

Screenshot 3: curl -v baseline showing HTTP 200 OK from the allowed client

6. Step 3 — Allowed HTTP Capture

```
$ sudo tcpdump -i [IFACE] host [ALLOWED-CLIENT-IP] and port 80  
-w allowed_http.pcapng  
  (from the allowed client, in a separate session)  
$ curl -v http://[SERVER-IP]/  
$ sha256sum allowed_http.pcapng
```

Evidence item	Value
Capture file	allowed_http.pcapng
SHA-256 hash	7725c105dc92cb6e5c41c9ac3b2419a383050bac7c679cf91d974c62e692ec30
Handshake observed	SYN → SYN-ACK → ACK → HTTP GET → 200 OK → FIN [confirm]
Capture duration / packet count	~2 seconds / 10 packets



Screenshot 4: Wireshark view of the allowed capture showing full TCP handshake and HTTP 200 OK

7. Step 4 — Inserting the Narrow DROP Rule

Only a single, source-specific rule was added, scoped to the assigned blocked client and TCP port 80, to avoid impacting any other traffic on the host.

```
$ sudo iptables -A INPUT -s [BLOCKED-CLIENT-IP] -p tcp --dport 80 -j DROP
$ sudo iptables -L INPUT -v -n --line-numbers
```

Field	Value
Rule added	-A INPUT -s [BLOCKED-CLIENT-IP] -p tcp -dport 80 -j DROP
Line number in INPUT chain	1
Initial packet counter	0
Initial byte counter	0

```
(kali㉿kali)-[~]
└─$ sudo iptables -A INPUT -s 192.168.199.135 -p tcp --dport 80 -j DROP
sudo iptables -L INPUT -v -n --line-numbers
[sudo] password for kali:
Chain INPUT (policy ACCEPT 346 packets, 46231 bytes)
num  pkts bytes target    prot opt in     out     source               destination           tcp dpt:80
1      0      0 DROP      tcp  --  *      *       192.168.199.135      0.0.0.0/0              tcp dpt:80
(kali㉿kali)-[~]
└─$
```

Screenshot 5: iptables -L INPUT -v -n --line-numbers showing the new rule with zeroed counters

8. Step 5 — Blocked Attempt: Capture and Counters

```
$ sudo tcpdump -i [IFACE] host [BLOCKED-CLIENT-IP] and port 80 -w blocked_http.pcapng
(from the blocked client, in a separate session)
$ curl -v --max-time 10 http://[SERVER-IP]/
```

```
$ sha256sum blocked_http.pcapng
$ sudo iptables -L INPUT -v -n
```

Evidence item	Value
Capture file	blocked_http.pcapng
SHA-256 hash	982bc0a807a6fadf6a8f461f0aeef7a898623e0d76449053313dbddf64c0816a
curl result	curl: (28) Connection timed out after 10001 milliseconds
SYN retransmissions observed in capture	7
DROP rule packet counter (after test)	7
DROP rule byte counter (after test)	420

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ curl -v --max-time 10 http://192.168.199.135/  
* Trying 192.168.199.135:80 ...  
* Connection timed out after 10000 milliseconds  
* closing connection #0  
curl: (28) Connection timed out after 10000 milliseconds  
(kali@kali)-[~]  
$  
  
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ sudo tcpdump -i any host 192.168.199.135 and port 80 -w blocked_http.pcapng  
[sudo] password for kali:  
tcpdump: WARNING: any: That device doesn't support promiscuous mode  
(Promiscuous mode not supported on the "any" device)  
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes  
^C7 packets captured  
14 packets received by filter  
0 packets dropped by kernel  
(kali@kali)-[~]  
$ sudo iptables -L INPUT -v -n --line-numbers  
sha256sum blocked_http.pcapng | tee blocked_http.sha256  
Chain INPUT (policy ACCEPT 426 packets, 58818 bytes)  
num  pkts bytes target    prot opt in     out     source                destination  
1      7   420 DROP      tcp  --  *      *       192.168.199.135      0.0.0.0/0            tcp dpt:80  
982bc0a807a6fadf6a8f461f0aeef7a898623e0d76449053313dbddf64c0816a  blocked_http.pcapng  
(kali@kali)-[~]  
$
```

Screenshot 6: Wireshark view of the blocked capture showing repeated unanswered SYN packets

```
(kali@kali)-[~]  
$ sudo iptables -L INPUT -v -n --line-numbers  
sha256sum blocked_http.pcapng | tee blocked_http.sha256  
Chain INPUT (policy ACCEPT 426 packets, 58818 bytes)  
num  pkts bytes target    prot opt in     out     source                destination  
1      7   420 DROP      tcp  --  *      *       192.168.199.135      0.0.0.0/0            tcp dpt:80  
982bc0a807a6fadf6a8f461f0aeef7a898623e0d76449053313dbddf64c0816a  blocked_http.pcapng  
(kali@kali)-[~]  
$
```

Screenshot 7: iptables -L INPUT -v -n showing incremented packet/byte counters on the DROP rule

Source	Count
SYN packets in blocked_http.pcapng	7

Source	Count
iptables DROP rule packet counter	7
Match? (Y/N)	Y

9. Step 6 — Allowed vs Blocked Comparison

Criterion	Allowed session (allowed_http.pcapng)	Blocked session (blocked_http.pcapng)
TCP handshake	Completed (SYN, SYN-ACK, ACK)	Never completed — no SYN-ACK returned
HTTP response	200 OK received	None — request never reached the application layer
Client behaviour	Single connection attempt, normal close (FIN)	Repeated SYN retransmissions with increasing backoff until client timeout
curl outcome	Success, page content returned	Timeout / connection error
Firewall counter activity	No matching DROP rule — INPUT ACCEPT/default policy applied	DROP rule packet/byte counters incremented on each retry

10. Step 7 — Rule Removal and Restoration

```
$ sudo iptables -D INPUT -s [BLOCKED-CLIENT-IP] -p tcp --dport 80 -j DROP
$ sudo iptables -L INPUT -v -n
$ curl -v http://[SERVER-IP]/ # from the previously blocked client
$ sudo iptables-save > ruleset_restored.rules
$ sha256sum ruleset_restored.rules
$ diff ruleset_original.rules ruleset_restored.rules
```

Evidence item	Value
Rule removed	Confirmed absent from iptables -L INPUT -v -n output
Restored ruleset file	ruleset_restored.rules
Restored ruleset SHA-256	[INSERT HASH]
diff against original	[INSERT — expected: no differing rules]
Post-removal curl result (blocked client)	[INSERT — expected: HTTP 200 OK]

```
(kali@kali)-[~]
$ sudo iptables-restore < ruleset_original.rules
sudo iptables -L INPUT -v -n --line-numbers
sha256sum ruleset_original.rules
[sudo] password for kali:
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
5820c9b3d903fc3962b36bc74867f02a7909e50ad46b09e80088043f4694045d  ruleset_original.rules

(kali@kali)-[~]
$
```

Screenshot 8: iptables -L INPUT -v -n showing restored empty chain and zeroed counters slot. It displays the clean INPUT chain and SHA-256 hash confirmation

11. Forensic Interpretation and Discussion

The iptables DROP rule silently discards incoming packets without returning an ICMP error or TCP Reset packet to the source. Because the client receives no acknowledgment, its TCP stack assumes packet loss and repeatedly retransmits the initial SYN packet with backoff until a timeout occurs. During the test, the kernel recorded exactly 7 SYN attempts and 420 bytes, matching the 7 captured frames in blocked_http.pcapng. This 1:1 reconciliation confirms every dropped packet was accounted for by netfilter and captured.

Using REJECT instead of DROP would cause the host to immediately respond with a TCP RST or ICMP Port Unreachable message. Consequently, curl would abort instantly with a Connection refused error rather than waiting out the timeout, and the capture would show a single SYN followed immediately by RST without retransmissions. Scoping the rule to the specific source IP and port 80 was crucial to avoid disturbing other lab traffic, ensuring administrative access remained unaffected.

12. Conclusion

This lab successfully demonstrated the practical implementation and forensic verification of host-based packet filtering using iptables. Baseline network operations and HTTP connectivity were verified, followed by the deployment of a targeted DROP rule for TCP port 80. Packet capture analysis and netfilter counter logs confirmed that the firewall silently dropped unauthorized connection attempts, resulting in TCP retransmission behavior and connection timeouts rather than immediate termination. All evidence files were cryptographically secured with SHA-256 hashes, cross-referenced against kernel counters, and finally, the system ruleset was completely restored to its original baseline state and verified.

